

Министерство цифрового развития

Федеральное государственное общеобразовательное учреждение высшего
образования

«Сибирский Государственный Университет Телекоммуникаций и
Информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Отчет по практической работе №5.

«Практическая работа 5. Абстрактный тип данных (ADT) р - ичное число»

Работу выполнил студент группы ИП213:

Жуйков С. А.

Терновский Д. Р.

Работу проверил ст. преподаватель кафедры ПМиК:

Агалаков А. А.

г. Новосибирск, 2025 г.

Цель работы

Сформировать практические навыки реализации абстрактного типа данных с помощью классов языка программирования Python и их модульного тестирования.

Задание

1. Реализовать абстрактный тип данных «р-ичное число», используя класс, в соответствии с представленной спецификацией.
2. Протестировать каждую операцию, определенную на типе данных по критерию С2, используя средства модульного тестирования.
3. Если необходимо, предусмотрите возбуждение исключительных ситуаций.

Спецификация типа данных «р-ичное число»

Данные:

Р-ичное число (тип `TPNumber`) - это действительное число (n) со знаком в системе счисления с основанием (b) в диапазоне 2..16, содержащее целую и дробную части. Точность представления числа – (с ≥ 0). Р-ичные числа неизменяемые.

Операции:

Операции могут вызываться только объектом р-ичное число (тип `TPNumber`), указатель на который в них передаётся по умолчанию. При описании операций этот объект называется `this` «само число».

метод «КонструкторЧисло»	
Вход:	Вещественное число (a), основание системы счисления (b), точность представления числа (c)
Предусловия:	Основание системы счисления (b) должно принадлежать интервалу [2..16], точность представления числа c ≥ 0 .
Процесс:	Инициализирует поля объекта <code>this</code> р-ичное число: система счисления (b), точность представления (c). В поле (n) числа заносится (a). Например: <code>TPNumber(a,3,3)</code> = число a в системе счисления 3 с тремя разрядами после троичной точки. <code>TPNumber(a,3,2)</code> = число a в системе счисления 3 с двумя разрядами после троичной точки.
Постусловия:	Объект инициализирован начальными значениями.
Выход:	Нет.
метод «КонструкторСтрока»	
Вход:	Строковые представления: р-ичного числа (a), основания системы счисления(b), точности предоставления число (c).
Предусловия:	Основание системы счисления (b) должно принадлежать интервалу [2..16], точность предоставления числа c ≥ 0 .
Процесс:	Инициализирует поля объекта <code>this</code> р-ичное число: основание системы счисления (b),

	точностью представления (с). В поле (n) числа this заносится результат преобразования строки (a) в числовое представление. b-ичное число (a) и основание системы счисления (b) представлены в формате строки. Например: TPNumber ("20","3","6") = 20 в системе счисления 3, точность 6 знаков после запятой. TPNumber ("0","3","8") = 0 в системе счисления 3, точность 8 знаков после запятой.
Постусловия:	Объект инициализирован начальными значениями.
Выход:	Нет
метод «Сложить»	
Вход:	P-ичное число d с основанием и точностью такими же, как у самого числа this.
Предусловия:	Нет.
Процесс:	Создаёт и возвращает p-ичное число (тип TPNumber), полученное сложением полей (n) самого числа this и числа d.
Выход:	P-ичное число.
Постусловия:	Нет.
метод «Умножить»	
Вход:	P-ичное число d с основанием и точностью такими же, как у самого числа this.
Предусловия:	Нет.
Процесс:	Создаёт и возвращает p-ичное число (тип TPNumber), полученное умножением полей (n) самого числа this и числа d.
Выход:	P-ичное число (тип TPNumber).
Постусловия:	Нет.
метод «Вычесть»	
Вход:	P-ичное число d с основанием и точностью такими же, как у самого числа this.
Предусловия:	Нет.
Процесс:	Создаёт и возвращает p-ичное число (тип

	TPNumber), полученное вычитанием полей (n) самого числа this и числа d.
Выход:	Р-ичное число (тип TPNumber).
Постусловия:	Нет.
метод «Делить»	
Вход:	Р-ичное число d с основанием и точностью такими же, как у самого числа.
Предусловия:	Поле (n) числа (d) не равно 0.
Процесс:	Создаёт и возвращает р-ичное число (тип TPNumber), полученное делением полей (n) самого числа this на поле (n) числа d.
Выход:	Р-ичное число (тип TPNumber).
Постусловия	Нет.
метод «Обратить»	
Вход:	Нет.
Предусловия:	Поле (n) самого числа не равно 0.
Процесс:	Создаёт р-ичное число, в поле (n) которого заносится значение, полученное как $1/(n)$ самого числа this.
Выход:	Р-ичное число (тип TPNumber).
Постусловия:	Нет.
метод «Квадрат»	
Вход:	Нет.
Предусловия:	Нет.
Процесс:	Создаёт р-ичное число, в поле (n) которого заносится значение, полученное как квадрат поля (n) самого числа this.
Выход:	Р-ичное число (тип TPNumber).
Постусловия:	Нет.
метод «ВзятьРЧисло»	
Вход:	Нет.
Предусловия:	Нет.
Процесс:	Возвращает значение поля (n) самого числа this.
Выход:	Вещественное значение.

Постусловия:	Нет.
метод «ВзятьРСтрока»	
Вход:	Нет.
Предусловия:	Нет.
Процесс	Возвращает р-ичное число (q) в формате строки, изображающей значение поля (n) самого числа this в системе счисления (b) с точностью (c).
Выход:	Строка.
Постусловия:	Нет.
метод «ВзятьОснованиеЧисло»	
Вход:	Нет.
Предусловия:	Нет.
Процесс:	Возвращает значение поля (b) самого числа this.
Выход:	Целочисленное значение.
Постусловия:	Нет.
метод «ВзятьОснованиеСтрока»	
Вход:	Нет.
Предусловия:	Нет.
Процесс:	Возвращает значение поля (b) самого числа this в формате строки, изображающей (b) в десятичной системе счисления.
Выход:	Строка.
Постусловия:	Нет.
метод «ВзятьТочностьЧисло»	
Вход:	Нет.
Предусловия:	Нет.
Процесс:	Возвращает значение поля (c) самого числа this.
Выход:	Целое значение.
Постусловия:	Нет.
метод «ВзятьТочностьСтрока»	

Вход:	Нет.
Предусловия:	Нет.
Процесс:	Возвращает значение поля (с) самого числа this в формате строки, изображающей (с) в десятичной системе счисления.
Выход:	Строка.
Постусловия:	Нет.

Листинг программы

TPNumber.py:

```
1  from __future__ import annotations
2
3  class TPNumber:
4
5      def __init__(self, a: float, b: int, c: int):
6          if (b < 2 or b > 16):
7              raise ValueError("arg 'b' must be in [2..16].")
8
9          if (c < 0): raise ValueError("arg 'c' must be >= 0.")
10
11         self.__num = a
12         self.__base = b
13         self.__accuracy = c
14
15
16     @classmethod
17     def from_strings(cls, a: str, b: str, c: str):
18         return cls(float(a), int(b), int(c))
19
20
21     def copy(self) -> TPNumber:
22         return TPNumber(self.__num, self.__base, self.__accuracy)
23
24
25     def add(self, d: TPNumber) -> TPNumber:
26
27         if (self.__base != d.__base): raise ValueError("'base' must be same.")
28         accuracy = max(self.__accuracy, d.__accuracy)
29
30         return TPNumber(self.__num + d.__num, self.__base, accuracy)
31
32
33     def multiply(self, d: TPNumber) -> TPNumber:
34
35         if (self.__base != d.__base): raise ValueError("'base must be same.'")
36         accuracy = max(self.__accuracy, d.__accuracy)
37
38         return TPNumber(self.__num * d.__num, self.__base, accuracy)
39
40
41     def subtract(self, d: TPNumber) -> TPNumber:
42
43         if (self.__base != d.__base): raise ValueError("'base must be same.'")
44         accuracy = max(self.__accuracy, d.__accuracy)
45
46         return TPNumber(self.__num - d.__num, self.__base, accuracy)
47
48
49     def divide(self, d: TPNumber) -> TPNumber:
50
51         if (d.__num == 0): raise ValueError("the field 'n' of the number 'd must not
52 be 0.")
53         if (self.__base != d.__base): raise ValueError("'base must be same.'")
54         accuracy = max(self.__accuracy, d.__accuracy)
55
56         return TPNumber(self.__num / d.__num, self.__base, accuracy)
57
58     def reverse(d: TPNumber) -> TPNumber:
59         if (d.__num == 0): raise ValueError("the field 'n' of the number 'd must not
60 be 0.")
61
62         return TPNumber(1 / d.__num, d.__base, d.__accuracy)
```

```

62
63     def square(self) -> TPNumber:
64         return TPNumber(self.__num ** 2, self.__base, self.__accuracy)
65
66     @property
67     def p_num(self) -> float:
68         return self.__num
69
70     @property
71     def p_str(self) -> str:
72         return str(self.__num)
73
74     @property
75     def base_num(self) -> int:
76         return self.__base
77
78     @property
79     def base_str(self) -> str:
80         return str(self.__base)
81
82     @property
83     def accuracy_num(self) -> int:
84         return self.__accuracy
85
86     @property
87     def accuracy_str(self) -> str:
88         return str(self.__accuracy)

```

test_TPNumber.py:

```

1  import pytest
2  from TPNumber import TPNumber
3
4
5  class TestTPNumber:
6
7      def test_init_base_too_low(self):
8          with pytest.raises(ValueError):
9              TPNumber(1.0, 1, 0)
10
11      def test_init_base_too_high(self):
12          with pytest.raises(ValueError):
13              TPNumber(1.0, 17, 0)
14
15      def test_init_negative_accuracy(self):
16          with pytest.raises(ValueError):
17              TPNumber(1.0, 10, -1)
18
19
20      @pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
21          (1.0, 10, 2, 2.0, 10, 3),
22          (5.5, 2, 1, 4.5, 2, 4),
23      ])
24      def test_add(self, a1, b1, c1, a2, b2, c2):
25          n1 = TPNumber(a1, b1, c1)
26          n2 = TPNumber(a2, b2, c2)
27
28          result = n1.add(n2)
29
30          assert result.p_num == a1 + a2
31          assert result.base_num == b1
32          assert result.accuracy_num == max(c1, c2)
33
34
35      @pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
36          (1.0, 10, 2, 2.0, 10, 3),
37          (5.5, 2, 1, 4.5, 2, 4),

```

```

38     ])
39     def test_multiply(self, a1, b1, c1, a2, b2, c2):
40         n1 = TPNumber(a1, b1, c1)
41         n2 = TPNumber(a2, b2, c2)
42
43         result = n1.multiply(n2)
44         assert result.p_num == a1 * a2
45         assert result.base_num == b1
46         assert result.accuracy_num == max(c1, c2)
47
48
49     @pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
50         (1.0, 10, 2, 2.0, 10, 3),
51         (5.5, 2, 1, 4.5, 2, 4),
52     ])
53     def test_subtract(self, a1, b1, c1, a2, b2, c2):
54         n1 = TPNumber(a1, b1, c1)
55         n2 = TPNumber(a2, b2, c2)
56
57         result = n1.subtract(n2)
58         assert result.p_num == a1 - a2
59         assert result.base_num == b1
60         assert result.accuracy_num == max(c1, c2)
61
62
63     @pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
64         (1.0, 10, 2, 2.0, 10, 3),
65         (5.5, 2, 1, 4.5, 2, 4),
66     ])
67     def test_divide(self, a1, b1, c1, a2, b2, c2):
68         n1 = TPNumber(a1, b1, c1)
69         n2 = TPNumber(a2, b2, c2)
70
71         result = n1.divide(n2)
72         assert result.p_num == a1 / a2
73         assert result.base_num == b1
74         assert result.accuracy_num == max(c1, c2)
75
76
77     @pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
78         (5.0, 10, 2, 0.0, 10, 2),
79         (5.0, 8, 2, 0.0, 8, 2)
80     ])
81     def test_divide_with_zero_num(self, a1, b1, c1, a2, b2, c2):
82         with pytest.raises(ValueError):
83             n1 = TPNumber(a1, b1, c1)
84             n2 = TPNumber(a2, b2, c2)
85
86             resn = n1.divide(n2)
87
88
89     @pytest.mark.parametrize("a, b, c", [
90         (1.0, 10, 2),
91         (5.5, 2, 1),
92     ])
93     def test_reverse(self, a, b, c):
94         n = TPNumber(a, b, c)
95
96         result = n.reverse()
97         assert result.p_num == 1 / a
98         assert result.base_num == b
99         assert result.accuracy_num == c
100
101
102     @pytest.mark.parametrize("a, b, c", [
103         (0.0, 10, 2)
104     ])
105     def test_reverse_with_zero_num(self, a, b, c):
106         with pytest.raises(ValueError):

```

```
107         n = TPNumber(a, b, c)
108         res = n.reverse()
109
110
111 if __name__ == "__main__":
112     pytest.main([__file__])
```

Тестовые наборы данных

```
@pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
    (1.0, 10, 2, 2.0, 10, 3),
    (5.5, 2, 1, 4.5, 2, 4),
])
def test_add(self, a1, b1, c1, a2, b2, c2):
    n1 = TPNumber(a1, b1, c1)
    n2 = TPNumber(a2, b2, c2)

    result = n1.add(n2)

    assert result.p_num == a1 + a2
    assert result.base_num == b1
    assert result.accuracy_num == max(c1, c2)
```

Рис. 1 — Пример набора данных для test_add.

```
@pytest.mark.parametrize("a, b, c", [
    (1.0, 10, 2),
    (5.5, 2, 1),
])
def test_reverse(self, a, b, c):
    n = TPNumber(a, b, c)

    result = n.reverse()
    assert result.p_num == 1 / a
    assert result.base_num == b
    assert result.accuracy_num == c
```

Рис. 2 — Пример набора данных для test_reverse.

```

@pytest.mark.parametrize("a1, b1, c1, a2, b2, c2", [
    (5.0, 10, 2, 0.0, 10, 2),
    (5.0, 8, 2, 0.0, 8, 2)
])
def test_divide_with_zero_num(self, a1, b1, c1, a2, b2, c2):
    with pytest.raises(ValueError):
        n1 = TPNumber(a1, b1, c1)
        n2 = TPNumber(a2, b2, c2)

        resn = n1.divide(n2)

```

Рис. 3 — Набор тестовых данных для метода test_divide_with_zero_num.

```

@pytest.mark.parametrize("a, b, c", [
    (0.0, 10, 2),
    (0.0, 8, 3)
])
def test_reverse_with_zero_num(self, a, b, c):
    with pytest.raises(ValueError):
        n = TPNumber(a, b, c)
        res = n.reverse()

```

Рис. 4 — Набор тестовых данных для метода test_reverse_with_zero_num.

```

def test_init_base_too_low(self):
    with pytest.raises(ValueError):
        TPNumber(1.0, 1, 0)

def test_init_base_too_high(self):
    with pytest.raises(ValueError):
        TPNumber(1.0, 17, 0)

def test_init_negative_accuracy(self):
    with pytest.raises(ValueError):
        TPNumber(1.0, 10, -1)

```

Рис. 5 — Набор тестовых данных для метода test_init.